

Appendix

10 Proofs

10.1 Safety

THEOREM 4 (AGREEMENT). *Well-behaved processes don't commit forking blocks.*

PROOF. Immediate from Lemma 7 and Lemma 8. $\square$

LEMMA 7 (SINGLE-VIEW AGREEMENT). *No pair of well-behaved processes commit different blocks in the same view.*

PROOF. Consider a pair of well-behaved processes that commit a block. A well-behaved process commits a block (at line 41) only after receiving Commit messages from a quorum of processes (at line 40). There is a well-behaved process in the intersection of every pair of quorums. Using the flag *c-sent* (at line 30 and line 34), a well-behaved process sends at most one Commit message in a view. Therefore, the two committed blocks are the same. $\square$

LEMMA 8 (TOTAL ORDER). *Every block committed in a view is an extension of the latest committed block in previous views.*

PROOF. A well-behaved process commits a block b (at line 41) only after receiving Commit messages from $2f + 1$ processes (at line 40), which includes at least one well-behaved process. A well-behaved process sends a Commit message (at line 31) only after receiving Prepare messages from $2f + 1$ processes (at line 30), which includes at least one well-behaved process. Thus, a well-behaved process sent a Prepare message for b . Thus, by Lemma 9 (part 1), b is an extension of the last committed block. $\square$

LEMMA 9 (INVARIANTS). *If the last committed block is b° from view v° , then*

- (1) *If a well-behaved process sends a Prepare message for a block b then b extends b° , i.e., $b^\circ \sqsubseteq b$, and*
- (2) *For every set of collected prepared blocks collection , and every set of processes P of size $2f + 1$ that respects the dependencies deps , let b_m be the block with the largest view v_m in $\text{collection}|_P$, i.e., $\langle v_m, b_m \rangle := \max(\text{collection}|_P)$, then b_m extends b° , i.e., $b^\circ \sqsubseteq b_m$, and*
- (3) *There are at least $f + 1$ well-behaved processes whose prepared block b_p is an extension of b° and whose prepared view v_p is greater than or equal to v° , i.e., $b^\circ \sqsubseteq b_p$ and $v^\circ \leq v_p$.*

PROOF. Consider the last view v° where a block b° is committed. We prove the invariants (1) to (3) in order by mutual induction on views.

Invariant 1. Now, we prove invariant (1): if a well-behaved process sends a Prepare message for b then b extends b° , i.e., $b^\circ \sqsubseteq b$. A well-behaved process p sends a Prepare message at two locations in the protocol: the fast path (at line 27) or the slow path (at line 71). Accordingly, we consider two cases.

Case 1 (the fast path): the process p sent a Prepare message at line 27. By the preceding condition (at line 26), the view v_c of p is the immediately previous view $v^* - 1$. By Lemma 10, p committed b_c in $v^* - 1$. Thus, b_c is the last committed block b° . Further, a condition before sending the Prepare message (at line 26) is $b_c = \text{prev-block}(b)$ which implies that $b_c \sqsubseteq b$. Therefore, $b^\circ \sqsubseteq b$.

Case 2 (the slow path): the process p sent a Prepare message for b_m at line 71. By the condition before sending the Prepare message (at line 70), b_m is the block with the maximum view in a *collection* of size $2f + 1$. Therefore, By the induction hypothesis for invariant (2) above, $b^\circ \sqsubseteq b_m$. Further, another condition before sending the Prepare message (line 70), is $b_m = \text{prev-block}(b)$. Therefore, $b^\circ \sqsubseteq b$.

Invariant 2. Now, we address invariant (2). Consider a *collection* of prepared blocks from $2f + 1$ processes P . Let b_m be the block with the maximum view v_m in $\text{collection}|_P$, i.e., $\langle v_m, b_m \rangle := \max(\text{collection}|_P)$. We prove that then b_m extends b° .

The pair of view and block $\langle v_m, b_m \rangle$ from a process p_m is in the *collection*. A pair $\langle v_m, b_m \rangle$ is collected (at line 65) only after receiving Ready or CondReady messages from at least $2f + 1$ processes (at line 64) which include a well-behaved process. A well-behaved process sends a Ready or CondReady message (at line 51 or line 54) only after receiving Echo messages from $2f + 1$ processes, or receiving CondReady or Ready messages from $f + 1$ processes (at line 56). Therefore, at least one well-behaved process p has sent the CondReady or Ready message after receiving $2f + 1$ Echo messages. We consider two cases.

Case 1 (at line 54): the process p sent a CondReady message. The condition that precedes sending a CondReady message for b_m (at line 53) is $v_m < v_p$. Further, When $\langle v_m, b_m \rangle$ from a CondReady is collected (at line 58) the dependency from p_m to p is added to deps . The $\text{collection}|_P$ respects deps . Thus, $\langle v_p, b_p \rangle$ from p is in the $\text{collection}|_P$ as well. Thus, since v_p is larger than v_m , the maximum view in the $\text{collection}|_P$ is not v_m .

Case 2 (at line 51): the process p sent a Ready message. The condition that precedes sending a Ready message (at line 50) implies that $\langle v_m, b_m \rangle \in ps$. Since $\langle v_m, b_m \rangle \in ps$, the well-behaved process p has sent a Prepare message for b_m in view v_m .

Next, we show that $v^\circ \leq v_m$. We show that there is a pair $\langle v, b \rangle$ in the *collection* such that $v^\circ \leq v$. Since the set P has $2f + 1$ processes, it contains at least $f + 1$ well-behaved processes P_1 . By Lemma 11, the collected view and block for each well-behaved process in P_1 is its own own prepared view v_p and blocks b_p . By the induction hypothesis for invariant (3) above, the prepared view of at least $f + 1$ well-behaved processes P_2 is greater than or equal to v° , i.e., $v^\circ \leq v_p$. There is at least one well-behaved process in the intersection of P_1 and P_2 . Thus, the *collection* contains at least one block with view v such that $v^\circ \leq v$.

Thus, the well-behaved process p has sent a Prepare message for b_m in view v_m which is greater than or equal to v° . Therefore, by the induction hypothesis for invariant (1) above, b_m extends b° .

Invariant 3. Now, we prove invariant (3): there are at least $f + 1$ well-behaved processes with prepared block b_p and view v_p such that $b^\circ \sqsubseteq b_p$ and $v^\circ \leq v_p$.

A well-behaved process committed block b° in view v° . A well-behaved process commits a block b° (at line 41) only after receiving Commit messages from $2f + 1$ processes (at line 40), which includes at least $f + 1$ well-behaved processes. A well-behaved process sends a Commit message (at line 31) only after storing b° in its b_p , and v° in its v_p (at line 32). Thus, at view v° , at least $f + 1$ well-behaved processes have their b_p set to b° and their v_p set to v° .

Now we consider views after v° , and show that the b_p of a well-behaved process can be only updated to extensions of b° . A well-behaved process assigns a block b to b_p at three locations in the

protocol (at line 32, or line 42). We show that b is an extension of b° . We consider three cases.

Case 1 (at line 32): The prepared block b_p is assigned the block b after receiving Prepare messages for b from $2f + 1$ processes (at line 30), which includes at least one well-behaved process. A well-behaved process p sent a Prepare message for b . By the induction hypothesis for the invariant (1) above, b is an extension of b° .

Case 2 (at line 37): The prepared block b_p is assigned the block b after receiving Commit messages for b from $f + 1$ processes (at line 35), which includes at least one well-behaved process. A well-behaved process sent a Commit message which can be tracked back to a well-behaved process that receives Prepare messages from $2f + 1$ processes (at line 30). Then the proof follows the same path as the previous case.

Case 3 (at line 42): The prepared block b_p is assigned the block that is being committed in the current view which is the trivially latest b° . Thus, b is equal to the last committed block b° . $\square$

LEMMA 10. *The last block that a well-behaved process has committed is its block b_c in the view v_c .*

PROOF. In the entire protocol, a well-behaved process updates its committed block b_c and view v_c (at line 42) if and only if it commits b_c in v_c (at line 41). $\square$

LEMMA 11. *If a collection contains a pair for a well-behaved process p , then the pair is the prepared view and block $\langle v_p, b_p \rangle$ of p .*

PROOF. Consider a pair $\langle v, b \rangle$ in the collection. A pair $\langle v, b \rangle$ is collected (at line 65) only after receiving Ready or CondReady messages from at least $2f + 1$ processes (at line 64) which include a well-behaved process. A well-behaved process sends a Ready or CondReady message (at line 51 or line 54) only after receiving Echo from $2f + 1$ processes, or Ready from $f + 1$ processes (at line 56). Therefore, at least one well-behaved process p has sent the Ready or CondReady message after receiving $2f + 1$ Echo messages. There is at least one well-behaved process in $2f + 1$ processes that sent Echo messages. A well-behaved process sends an Echo message for a process p (at line 47) only after receiving the Send message from p (at line 46). If the sender p is well-behaved, it only sends its own prepared view and block v_p and b_p in a Send message (at line 45). $\square$

10.2 Liveness

LEMMA 12. *In every view after the first in GST, every well-behaved process commits a block.*

PROOF. Immediate from Lemma 13 and Lemma 14. $\square$

LEMMA 13. *In the first or second view following GST, every correct process commits a block.*

PROOF. Consider the first view after GST. We consider two cases: (a) A well-behaved process commits a block in this view. Then, the conclusion follows from Lemma 14. (b) No well-behaved process commits a block in this view. Then, the leader of this view times out, and a new view is started. The leader of the next view didn't commit a block in this view. Thus, the conclusion follows from Lemma 15 for the second view. $\square$

LEMMA 14. *After GST, if all well-behaved processes have committed a block in the previous view, then all well-behaved processes will eventually commit a block in the current view.*

PROOF. By single-view agreement of Lemma 7, every well-behaved process committed the same block b° in $v^* - 1$ (at line 41). Thus, each set its v_c and b_c to $v^* - 1$ and b° respectively (at line 42). The well-behaved leader of the current view receives a NewView event (at line 16). It makes a block b that extends b° , and proposes it (at line 22-23). When a well-behaved process p receives the Propose message (at line 26), since its own v_c is $v^* - 1$, and its $b_c = b^\circ$ is extended by the the proposed block b , the conditions (at line 26) hold, and p sends a Prepare message. Therefore, well-behaved processes eventually receive Prepare messages from each other. Thus, every well-behaved process eventually receives $2f + 1$ Prepare messages, and sends a Commit message (at line 30-31). Therefore, well-behaved processes eventually receive Commit messages from each other. Thus, every well-behaved process eventually receives $2f + 1$ Commit messages, and commits the block (at line 40-41). $\square$

LEMMA 15. *After GST, if the leader hasn't committed a block in the previous view, then all well-behaved processes commit a block in the current view.*

PROOF. The leader receives a NewView event (at line 16). If it hasn't committed a block in the previous round (at line 24), it sends a Collect message (at line 25). Every well-behaved process p receives the Collect message (at line 44), and sends its prepared view v_p and block b_p (at line 45) in a Send message. We show that every well-behaved process p_c will eventually receive and collect the prepared view v_p and block b_p of p .

Similar to the classical Bracha broadcast, well-behaved processes send an Echo message in response to a Send message, and send a Ready or CondReady message in response to an Echo message. The important difference is that when a well-behaved process p' receives Echo messages (at line 49), it sends a Ready or CondReady message only under certain conditions. We show that these conditions are satisfied in a set P' of at least $f + 1$ well-behaved processes.

By Lemma 16, since the prepared view and block of p are v_p and b_p respectively, a set P' of at least $f + 1$ well-behaved processes have sent a Prepare(v_p, b_p) message. We show that the conditions to send a Ready or CondReady message holds for every process

p' in P' . Let the prepared view of p' be v'_p . We consider two cases: Case (1): $v_p < v'_p$. The process p' sends a CondReady message (at line 53-54). Case (2): $v_p \geq v'_p$. Let the set of sent prepare messages of p' be ps' . By Lemma 18, $\langle v_p, b_p \rangle \in ps'$. Therefore, p' sends a Ready message (at line 53-54). Therefore, every well-behaved process p' in P' of size $f + 1$ sends a Ready or CondReady message.

We now consider all well-behaved processes p in the non-increasing order of their prepared view v_p . We prove by induction in the sequence that every well-behaved process p_c eventually adds $\langle v_p, b_p \rangle$ to its *collection*. Consider a well-behaved process p_c . (a) For the base case, consider the first process p with the largest v_p in well-behaved processes. Since no other well-behaved process p' has a larger v_p , no well-behaved process p' in P' sent a CondReady message. Every process p' sent a Ready message for b_p . Thus, p_c receives a Ready message for b_p from p' (at line 56) and adds p' to *readies*(p, v_p, b_p). Since the size of P' is at least $f + 1$, eventually $|readies(p, v_p, b_p)| \geq f + 1$ at p_c . Thus, through amplification (at line 61-63), each well-behaved process p_c sends a Ready message for b_p if it has not yet done so. Since there are at least $2f + 1$ well-behaved processes, each well-behaved process p_c receives at least $2f + 1$ Ready messages for b_p . Thus, eventually $|readies(p, v_p, b_p)| \geq 2f + 1$ at p_c . Thus, p_c eventually adds $\langle v_p, b_p \rangle$ to its *collection* (at line 65). (b) For the inductive case, consider a well-behaved process p with a prepared view v_p . Assume that the prepared view and block of every well-behaved process whose prepared view is less than v_p is already collected at p_c . Thus, if a process p' in P' has sent a CondReady message for b_p , the condition to receive CondReady (at line 58) is satisfied, p_c adds p' to its *readies*(p, v_p, b_p). If a process p' in P' has sent a Ready message for b_p , p_c simply receives it (at line 56), and adds p' to its *readies*(p, v_p, b_p). Since the size of P' is at least $f + 1$, eventually $|readies(p, v_p, b_p)| \geq f + 1$ at p_c . Thus, similar to the base case, after amplification, p_c eventually adds $\langle v_p, b_p \rangle$ to *collection*.

Since there are at least $2f + 1$ well-behaved processes p , every well-behaved process p_c including the leader will eventually have a *collection* of size at least $2f + 1$. Thus, the leader eventually takes the block with the maximum view, extends it and proposes it (at line 64-69). Every well-behaved process will eventually have a *collection* that includes the same set P' as the leader. Thus, every well-behaved process will eventually accept the block that the leader proposed, and send a Prepare message (at line 70-71).

Since there are at least $2f + 1$ well-behaved processes, every well-behaved process eventually receives $2f + 1$ Prepare messages, and sends a Commit message (at line 30-31). Similarly, since there are at least $2f + 1$ well-behaved processes, every well-behaved process will receive $2f + 1$ Commit messages, and commits the block (at line 40-41). $\square$

LEMMA 16. *If a well-behaved process has a prepared view v_p and block b_p , then at least $f + 1$ well-behaved processes have sent a Prepare(v_p, b_p) message.*

PROOF. A well-behaved process sets its prepared view v_p and block b_p at three locations in the protocol. We consider each in turn.

Case (1): line 32. A well-behaved process p sets its prepared view v_p and block b_p to v and b (at line 32) only after receiving

a $\text{Prepare}(v, b)$ message from $2f + 1$ processes (at line 30), and at least $f + 1$ of them are well-behaved.

Case (2): line 37. A well-behaved process p sets its prepared view v_p and block b_p to v and b (at line 37) only after receiving a $\text{Commit}(v, b)$ message from $f + 1$ processes (at line 35) which includes at least a well-behaved process. A well-behaved process that sends an amplifying Commit is tracked back to a well-behaved process that sends a non-amplifying Commit. Thus, a well-behaved process sent a Commit message after receiving a $\text{Prepare}(v, b)$ message from $2f + 1$ processes, and at least $f + 1$ of them are well-behaved.

Case (3): line 42. A well-behaved process p sets its prepared view v_p and block b_p to v and b (at line 42) only after receiving a $\text{Commit}(v, b)$ message from $2f + 1$ processes (at line 40), and at least one of them is well-behaved. A well-behaved process sends a $\text{Commit}(v, b)$ message (at line 31) only after receiving a $\text{Prepare}(v, b)$ message from $2f + 1$ processes (at line 30) and at least $f + 1$ of them are well-behaved. $\square$

LEMMA 17 (COMMIT TOTALITY). *After GST, if a well-behaved process commits a block, then every well-behaved process eventually commits that block.*

PROOF. A well-behaved process commits a block (at line 41) only after receiving a Commit message from $2f + 1$ processes (at line 40). At least $f + 1$ of them are well-behaved. Thus, every well-behaved process eventually receives the Commit message from at least $f + 1$ processes. If a well-behaved process receives $f +$

1 Commit message (at line 35), it amplifies it. Thus, every well-behaved process eventually receives a Commit message from every well-behaved process. Thus, every well-behaved process eventually receives $2f + 1$ Commit messages (at line 40). Therefore, every well-behaved process eventually commits the block (at line 41). $\square$

LEMMA 18. *Let v_p be the prepared view, and ps be the set of sent prepare messages of a well-behaved process p . If p has sent a $\text{Prepare}(v, b)$ message, and $v \geq v_p$, then $\langle v, b \rangle \in ps$.*

PROOF. The proof is by induction on reachable states. In the initial state, no message has been sent.

A well-behaved process sends a new $\text{Prepare}(v, b)$ message at two locations in the protocol (at line 27 and line 71). In both, it adds $\langle v, b \rangle$ to ps (at line 28 and line 72).

A well-behaved process p removes $\langle v, b \rangle$ pairs from its ps in three locations (at line 43 and line 33). For each, we show that $v < v_p$.

Case (1): A well-behaved process updates ps to $ps|_{v=v_p}$ (at line 43) only after updating its prepared block v_p to the latest view v^* (at line 42), i.e., $v_p = v^*$. Further, for any $\text{Prepare}(v, b)$ message that p has sent, trivially $v \leq v^*$. We consider two cases: Case (a): $v = v^*$. The pair with view v is not removed from ps . Case (b): $v < v^*$. Trivially $v < v_p$.

Case (2): A well-behaved process updates ps to $ps|_{v=v_p}$ (at line 38) only after updating its prepared block v_p to the latest view v^* (at line 37), i.e., $v_p = v^*$. Then, this case is similar to the previous case.

Case (3): A well-behaved process updates ps to $ps|_{v=v_p}$ (at line 33). It filters only pairs $\langle v, b \rangle$ where $v < v_p$. $\square$